

Domain-Sovereign Semantic-to-Metadata Mapping Agent Architecture

*System Design — An Agentic Extension of the Holonic Semantic Governance
Architecture (Level 2, Domain Holon Layer)*

Draft v1.2 | Prepared for Jeffrey Wallk, Senior Enterprise Architect & Knowledge Engineering Leader |
July 8, 2026

Executive Summary

This document specifies the agentic workflow that maps concepts in the federated GraphDB — the enterprise's component ontologies — to content metadata for digital artifacts: data structures, documents, business rules, constraints, process maps, events, measures, personas, and outcomes. It is a direct extension of the Level 2 Domain Holon layer of the Holonic Semantic Governance Architecture (v2.0): each domain already owns a component ontology, a domain-specific vector database, and a set of Policy Enforcement Points. This design adds a Domain Mapping Agent to every domain holon, and a governance pipeline that lets those agents propose mappings while the deterministic semantic layer — SHACL, SWRL, and PEPs — decides what is actually written to the graph.

The central design commitment, carried directly from your framing, is this: mapping agents are domain-sovereign, not general-purpose. Each agent is tuned and trained on exactly one domain's ontology, vocabulary, and content patterns, so it becomes an expert in one domain rather than mediocre across all of them. This is also how the architecture scales — as a domain's complexity grows, its agent (and its ontology) can be decomposed into sub-domain holons without touching any other domain's governance.

The second design commitment is architectural, not just procedural: these are agentic AI systems built on Claude, not Claude Skills invoked ad hoc. A Skill is a stateless, reusable procedure with no persistent identity or memory of its own performance over time. What you're describing — an agent that is individually tuned to a domain, monitored for drift, and subject to circuit-breaker isolation — requires a persistent, registered identity: a domain holon's agent, not a shared skill. Skills still have a role in this design, but as the reusable tool library each domain agent calls, not as the unit of domain expertise itself. Section 5 makes this decision explicit as a trade-off analysis.

Revision v1.1 answered what an agent looks like inside and which agents to build first: Section 3.1 specifies the agent as two component classes -- a reusable chassis and an isolated cartridge -- so that reuse and domain isolation are structural properties of the code, not just governance policy; Section 3.2 gives the initial roster of domain agents to build. Revision v1.2 (this one) answers a follow-up question: what exactly gets written to the graph. Section 3.7 specifies the mapping as three node types -- an artifact node, a domain concept node, and a reified mapping assertion node -- with typed, directed edges between them, and states precisely which node the 5W (WHO/WHAT/WHEN/WHERE/WHY) context binds to.

Core thesis: the mapping agents don't decide what is true — the ontology does. Every agent proposal passes through a deterministic validation gate before it is committed. This is what keeps hallucination and semantic drift measurable and bounded rather than silently accumulating in the knowledge graph.

1. Requirements

1.1 Functional Requirements

- Ingest a digital artifact of any of nine content types (data structure, document, business rule, constraint, process map, event, measure, persona, outcome) and propose a mapping from that artifact's content to one or more concepts in the federated GraphDB.

- Route each artifact to the domain(s) whose component ontology governs its subject matter, using the five-dimensional relevancy signature (WHO/WHAT/WHEN/WHERE/WHY) already established in the RDSG gateway.
- Support artifacts that legitimately span more than one domain (e.g., a process map touching both Finance and Operations) without forcing a single-domain assignment.
- Validate every proposed mapping deterministically against the target domain's SHACL shapes, SWRL rules, and PEPs before it is committed to the graph.
- Record every accepted mapping as a KGCL change record carrying full WHO/WHAT/WHEN/WHERE/WHY provenance, including which agent proposed it and which governance checks it passed.
- Measure, per domain agent, how often proposed mappings are accepted, rejected, or flagged — and make that signal actionable (retraining, circuit-breaker isolation) without requiring a system-wide shutdown.
- Allow a domain to be decomposed into sub-domain agents as its scope or content volume grows, without re-validating or disrupting other domains.

1.2 Non-Functional Requirements

Dimension	Target	Rationale
Latency — Tier 1 (operational)	< 5s per artifact, single-domain	Matches the existing Tier 1 Operational decision-tier SLA (>98% accuracy, real-time)
Latency — Tier 2/3 (tactical/strategic)	Minutes to hours, batch-tolerant	Full RCB assembly and cross-domain reconciliation are not latency-critical at these tiers
Accuracy (mapping acceptance rate)	Domain-specific baseline, tracked per agent	A single enterprise-wide accuracy target would mask which domain is degrading
Isolation	One agent's failure must not affect another domain's agent	Direct requirement from your domain-sovereignty framing; matches Circuit Breaker fault containment
Auditability	Every commit traceable to agent, ontology version, and validation result	KGCL is the architecture's existing change-of-record mechanism
Extensibility	New domain = new agent + ontology, no core pipeline change	Federated topology must not require central re-engineering per domain

1.3 Constraints and Assumptions

- The federated GraphDB and its component ontologies (per-domain OWL/SHACL/SWRL) already exist, per the Holonic Semantic Governance Architecture v2.0 supplied as source context.
- Domain-specific vector databases (Pinecone/Weaviate-class, per the existing L2 Domain Holon pattern) are assumed available for hybrid symbolic + vector retrieval within each domain.

- Agents are built using Claude (via the Claude Agent SDK / Model Context Protocol) rather than a third-party agent framework, per your stated preference.
- The nine content-type categories are treated as the initial artifact taxonomy; the design assumes this list will grow and is built to absorb new types without structural change (see Section 3.3).
- This document does not specify the underlying GraphDB vendor, embedding model, or specific SHACL shape libraries — those are implementation choices left open in Section 6.

2. High-Level Design

The pipeline is presented as a diagram alongside this document. It has seven stages: Federated GraphDB, Content & Context Router, Domain Mapping Agents, Deterministic Validation Gate, Cross-Domain Reconciliation, KGCL Commit, and a Drift & Governance Monitor that runs continuously alongside the agents and validation gate rather than as a discrete pipeline step.

Stage	Function	Holonic Counterpart
Federated GraphDB	System of record for all domain ontologies and their vector indexes	L1/L2 Knowledge & Domain Holons
Content & context router	Tags the incoming artifact with WHO/WHAT/WHEN/WHERE/WHY and routes by WHERE (domain)	RDSG Stage 0 — RCB assembly
Domain mapping agents	One Claude-based agent per domain holon proposes candidate concept mappings	L2 Domain Holon agent
Deterministic validation gate	SHACL structural check, SWRL inference check, PEP compliance check	RDSG Steps 1–3
Cross-domain reconciliation	Merges or links mappings for artifacts spanning more than one domain	STG — holonic composition
KGCL commit	Writes the governed mapping to the graph as a versioned, provenance-carrying change record	ORSD/KGCL change protocol
Drift & governance monitor	Continuously scores each agent's proposals (MTBH, Value-Drift, Intent Delta) and can trip a per-agent circuit breaker	MTBH / Circuit Breaker

Note the direction of authority: agents propose, the ontology decides. Nothing a Domain Mapping Agent produces is written to the graph until it clears the deterministic validation gate. This is the mechanism that answers your stated goal directly — using Claude for the parts that require judgment (reading unstructured content, recognizing which concept an artifact is about) while keeping the parts that require correctness (is this mapping actually valid) fully deterministic and outside the LLM's control.

3. Deep Dive

3.1 Agent Component Design: Chassis and Cartridge

Each domain holon that currently has a component ontology, a vector database, and a set of PEPs is extended with one Domain Mapping Agent. To make reuse and isolation structural properties of the code -- not just governance policy that a developer has to remember to honor -- every agent is built from two component classes that are never allowed to blend into one codebase:

Reusable chassis (identical for every agent; owned centrally)

The chassis is written once, owned by the Semantic Governance Center of Excellence, and instantiated unchanged for every domain. A fix, a security patch, or a capability upgrade to any chassis component takes effect for every agent the next time that agent's cartridge is loaded -- no domain-by-domain rework.

Chassis Component	Role (identical across all agents)
Retrieval interface	Runs the two-level hybrid search (vector ANN narrows candidates, ontology confirms validity) against whichever domain vector DB and ontology endpoint the cartridge points it at
LLM reasoning client	Wraps the Claude Agent SDK call, assembles the prompt, and enforces the output schema -- the same client code for every domain; only the system prompt and few-shot pack it loads differ
MCP tool client	Standardized, auditable tool-calling interface to the domain's ontology and vector DB, per the architecture's existing MCP standard for agent-to-hierarchy communication
Proposal schema and parser	Validates the LLM's raw output against the MappingProposal schema before it is ever handed to the deterministic validation gate
Telemetry emitter	Reports every proposal's outcome (accepted / rejected / flagged) to the Drift and Governance Monitor in the common MTBH / Value-Drift format
Lifecycle state machine	Implements the shared DEV → STG → PRD → QUARANTINED transition rules from Section 3.9

Isolated cartridge (unique per domain; owned by that domain)

The cartridge is where “train and tune each agent using domain-specific knowledge” actually happens. It is authored and revised exclusively by that domain's own Governance Team and SMEs, is never merged with another domain's cartridge, and is the only thing that changes when a domain is retrained.

Cartridge Component	Role (unique to one domain)
Domain ontology binding	The specific SHACL shapes, SWRL rules, and PEP tags this agent is scoped to -- exactly one domain's slice of the federated GraphDB, per Section 1.1's routing requirement
Domain vector index / corpus	The embeddings and source documents indexed for this domain only
Domain tuning pack	The system prompt template, few-shot examples, and evaluation set used to tune this agent's reasoning -- authored by this domain's SMEs and never

Cartridge Component	Role (unique to one domain)
	shared with another domain
Agent identity and health state	The registry entry, and this agent's own MTBH / Value-Drift / Intent Delta history and circuit-breaker state -- never reset or influenced by another domain's activity

Scope, per this split: an agent cannot read or write outside its own cartridge's ontology binding; cross-domain content is handled by reconciliation (Section 3.5), never by widening one agent's scope. Grounding follows the two-level hybrid search built into the chassis's retrieval interface -- vector search narrows candidates, the ontology decides which are valid. Output is always a structured mapping proposal (candidate concept URI/IRI, confidence score, supporting evidence) -- never a direct graph write; the chassis gives no agent write capability of its own.

This is the concrete mechanism behind “reuse and isolation”: reuse lives entirely in the chassis (one implementation, six components, used by every agent unchanged), and isolation lives entirely in the cartridge (four components, one instance per domain, owned by that domain alone). Retraining Finance's tuning pack cannot touch Engineering's prompt, corpus, or health state, because they are different cartridge instances loaded into the same chassis code -- not different branches or forks of the agent itself.

3.2 Recommended Agent Roster

The following seven agents are the initial roster to build, one cartridge per domain your own architecture already names as an independent discipline (the KSWG engineering disciplines, plus the FP&A domain detailed in Sections 2.3 and 3 of the source architecture). Each row is one cartridge; all seven load into the same chassis.

Domain Agent	KSWG Pillar	Primary Artifact Categories Owned	Domain Governance Owner
Finance / FP&A	Domains	Business rules, constraints, process maps, measures, outcomes	FP&A Domain Governance Team
Requirements Engineering	Engineering	Data structures, constraints, documents	Requirements Domain Governance Team
Systems Engineering (MBSE)	Engineering	Data structures, process maps, events	MBSE Domain Governance Team
Systems Safety (STPA/STAMP)	Systems	Constraints, events, business rules	Systems Safety Domain Governance Team
Knowledge Systems	Engineering	Documents, data structures, personas	Knowledge Systems Domain Governance Team
Enterprise Architecture	Systems	Process maps, outcomes, measures	Architecture Domain Governance Team
Decision Analysis	Policy	Outcomes, measures, personas	Decision Analysis Domain Governance Team

Every one of the nine artifact categories is owned by at least one agent, and several are deliberately owned by more than one -- a business rule artifact routed to both Finance and Systems Safety is exactly the cross-domain case Section 3.5 reconciles, not an error. New

domains (a Legal agent, a Product Operations agent, and so on) are added the same way: a new cartridge, no chassis change, per the extensibility requirement in Section 1.2.

3.3 Content-Type to Metadata Category Mapping

The nine artifact categories map to existing ontological mechanisms already defined in the domain holon architecture, rather than requiring new governance machinery:

Artifact Category	Ontological Mechanism	Governing Holonic Level
Data structures	OWL/RDFS class + SHACL shape	L1 Knowledge Holon
Documents	DCAT catalog metadata + KGCL provenance	L1 Knowledge Holon
Business rules	SWRL rule + LKIF deontic obligation/permission	L1/L2, Deontic Holon
Constraints	SHACL shape (invariant) or LKIF prohibition	L1 Knowledge Holon
Process maps	Process-centric ontology class (FIBO-style) + SUDA tagging	L2 Domain Holon
Events	n-ary event relation (Relationship Management)	L1/L2, WHEN dimension
Measures	Metric/KPI class linked to the Value Stream Ontology	L4 Enterprise Holarchy
Personas	Persona-holon, WHO dimension, RBAC/PEP scope	L1/L2, WHO dimension
Outcomes	Value-stream / decision-tier outcome class	L4 Enterprise Holarchy

This table is the extension point for future content types: adding a tenth category means adding a row here and confirming which existing ontological mechanism it maps to — it does not require a new pipeline stage.

3.4 Deterministic Validation Gate

Every proposed mapping passes three checks, in order, before reconciliation. A failure at any check quarantines the proposal with a rejection record rather than silently dropping it:

1. SHACL structural validation — does the proposed concept's shape actually accept this artifact as a valid instance (required properties present, types correct)?
2. SWRL inference check — is the mapping logically consistent with what can already be inferred about this concept from the domain ontology? A mapping that would create a contradiction is rejected.
3. PEP compliance — does the mapping violate any Compliance, Risk, Legal, Security, Data Governance, or AI Governance policy attached to the domain?

This is the architecture's specific answer to the hallucination-management goal: an LLM-proposed mapping is treated as a hypothesis, never as ground truth. Ground truth is whatever the SHACL/SWRL layer says it is. A high-confidence but structurally invalid proposal is still rejected.

3.5 Cross-Domain Reconciliation and Holonic Decomposition

Artifacts that legitimately touch more than one domain (a process map that references both a Finance concept and an Operations concept) are fanned out to each relevant Domain Mapping Agent in parallel. Each agent proposes independently, within its own scope; reconciliation then produces a Mapping Ontology entry — the same bottom-up/top-down composition mechanism already used in the STG environment — that links the two domain-local mappings rather than merging them into one shared record.

As a domain grows in complexity, this same mechanism supports decomposition: a domain's agent and ontology can be split into sub-domain agents (for example, a Finance agent splitting into FP&A and Treasury agents) that inherit from the parent domain holon through the polymorphic inheritance protocol already defined in the KSWG Core-to-Engineering topology. No other domain's agents or governance are affected by this split.

3.6 KGCL Commit and Provenance

An accepted mapping is written as a KGCL change record carrying: WHAT (the concept-to-artifact link), WHY (agent proposal rationale and confidence score), WHO (the specific agent identity from the registry), WHEN (timestamp), and the validation result (which SHACL/SWRL/PEP checks it passed). This makes every mapping in the graph traceable back to the exact agent version and governance state that produced it — required for later drift analysis and for any audit of a specific mapping's origin.

3.7 Graph Data Model: Nodes, Relation, and 5W Binding

Section 3.6 describes the KGCL record's fields; this section specifies what those fields mean as graph structure. A mapping is not a single edge between an artifact and a concept -- it requires three node types, because a bare edge cannot carry the WHO/WHAT/WHEN/WHERE/WHY context every mapping must record.

Node Type	Identity	Key Properties
Artifact node	artifact_id, persistent once admitted	Content-type category (one of the nine, Section 3.3); an L1 Knowledge Holon in candidate/unmapped state until a mapping assertion resolves it
Domain concept node	Concept URI (e.g. fin:ForecastingProcess)	Owning domain, SHACL shape, SWRL/PEP bindings -- the existing L1/L2 concept the artifact is being resolved against
Mapping assertion node	Assertion id, minted per KGCL commit	The WHO/WHAT/WHEN/WHERE/WHY properties below -- this is the KGCL record from Section 3.6, modeled as a graph node rather than a flat log line

Two directed edges originate from the assertion node -- it is the hub, not an intermediate stop on a path from artifact to concept:

- assertion --about--> artifact -- which artifact this assertion concerns
- assertion --resolvesTo--> concept -- which domain concept it resolves the artifact to

This is the standard reification pattern: only a node can carry properties, so the relation itself must be a node whenever it needs context attached to it, which every mapping in this architecture does. Once an assertion passes the validation gate (Section 3.4), a denormalized

shortcut edge may also be materialized directly between the two entity nodes -- artifact -- mappedToConcept--> concept, inverse hasMappedArtifact -- so a simple traversal can answer “what concept is this artifact mapped to” without hopping through the assertion. That direction, artifact toward concept, is deliberate: it mirrors rdf:type (instance toward class). The artifact submits to the concept's governance; the concept does not belong to the artifact.

The 5W context binds to the assertion node only -- it is the same Requirement Context Bundle vector already defined for RDSG traversal (source architecture Section 3.3), scoped to one artifact-to-concept resolution:

Dimension	Value in this model	Holonic role
WHO	The proposing agent's identity (e.g. agent::finance)	Persona-holon
WHAT	The resolved concept URI (restated from the resolvesTo edge target for query convenience)	Concept-holon
WHEN	Timestamp of proposal and, if different, of validation	Temporal-holon
WHERE	The domain governing the target concept	Domain-holon boundary
WHY	Confidence, supporting evidence, and governance intent	Intent-holon

This introduces no new fields beyond what Section 3.6 already defines -- it specifies that those fields belong to a node with two typed outgoing edges, not a flat record. That is what makes the graph itself answerable with a traversal query (“which artifacts has this agent resolved,” “which concepts has this artifact touched over its lifetime”) rather than requiring a scan of the KGCL log.

3.8 Drift, Hallucination, and Circuit-Breaker Monitoring

Three metrics, already defined at the architecture level, apply directly to mapping agents without modification:

- MTBH (Mean Time Between Hallucinations), scored per agent and per WHO/WHAT/WHEN/WHERE/WHY dimension — a spike in rejected WHAT-dimension mappings for one agent does not trigger review of a different agent's WHERE-dimension routing.
- Value-Drift Coefficient — the distance between an agent's current mapping behavior and the governance contract encoded in its domain ontology at the time it was tuned. Rising drift signals the agent needs retraining against a refreshed domain corpus, not a system-wide pause.
- Intent Delta — the gap between what an agent believes an artifact is about and how that mapping is actually used downstream (e.g., a mapping that is repeatedly manually corrected by domain SMEs after commit).

When any of these metrics crosses a domain-specific threshold, the Circuit Breaker trips for that one agent only: its proposals are quarantined pending Domain Governance Team review, and — per the existing sanitization/anti-collapse principle — its rejected outputs are excluded from any future fine-tuning or few-shot corpus for that agent. No other domain agent is paused.

3.9 Agent Registry and Lifecycle

Every Domain Mapping Agent is entered in the existing Domain Agent Registry with a persistent identity, its bound domain scope, its current lifecycle state, and an immutable audit trail. New agents (and agent versions, after retraining) move through the same DEV/STG/PRD lifecycle already defined for domain holons: DEV for tuning against a domain corpus, STG for reconciliation testing against other domains' agents, PRD for live proposal generation against the production graph.

4. Scale and Reliability

4.1 Load Tiering

Decision Tier	Latency Target	Pipeline Behavior	Example
Tier 1 — Operational	< 5s	Pre-validated domain rules; skip full reconciliation when single-domain and high-confidence	Auto-tagging an incoming support ticket with a data-structure concept
Tier 2 — Tactical	Daily/weekly batch	Full validation gate and cross-domain reconciliation	Mapping a new process map spanning two domains
Tier 3 — Strategic	Weekly/monthly batch	Full holarchic governance, Enterprise Council visibility for cross-domain conflicts	Mapping outcome metrics into the Value Stream Ontology

4.2 Horizontal Scaling

Scaling this system is scaling the number of domain holons, not the throughput of a single shared agent. Because each Domain Mapping Agent is scoped to one domain and reads/writes only within it, adding a domain (or decomposing one into sub-domains, per 3.5) is an additive operation: a new cartridge is authored, tuned, registered, and moved through DEV/STG/PRD without any change to the shared chassis, the router, the validation gate, or any other agent's cartridge. This is the direct payoff of the chassis/cartridge split in Section 3.1: horizontal scale is a governance operation -- author one more cartridge -- not a re-architecture.

4.3 Failure Containment

Failure is contained at three levels: a rejected mapping never reaches the graph (validation gate); a drifting or hallucinating agent is isolated without pausing other agents (per-agent Circuit Breaker); and a domain-wide ontology change is versioned through KGCL so a bad change can be rolled back without affecting other domains' change history.

5. Trade-off Analysis

5.1 Domain-Sovereign Agents vs. Alternatives

Option	Complexity	Scalability	Assessment
A. Single centralized mapping agent (one model, all domains loaded)	Low to build, high to maintain	Poor — every new domain risks regressing existing ones; retraining is all-or-nothing	Rejected — directly conflicts with your domain-expertise requirement
B. Domain-sovereign agent federation (chosen)	Higher upfront (N agents to register and tune)	Strong — domains added or split independently	Selected — matches existing L2 Domain Holon pattern and your stated constraints
C. Pure symbolic mapping, no LLM	Low runtime risk, high authoring cost	Poor for unstructured content (documents, free-text business rules)	Rejected as sole approach — reasoning over unstructured artifacts needs an LLM; used instead as a validation layer under Option B

5.2 Agents vs. Skills for Domain Expertise

You raised this directly: should these be framed as Claude Skills, or as agentic AI built on Claude? The two are not mutually exclusive, and this design uses both, at different layers:

- A Skill is the right unit for a reusable procedure with no need for persistent identity — 'extract candidate concepts from a process map,' 'summarize a business rule into deontic form.' These are stateless, callable the same way regardless of which domain invokes them.
- A registered, domain-scoped agent is the right unit for domain expertise itself — because expertise here means persistent tuning, a tracked accuracy history, drift scoring, and the ability to be individually circuit-broken. A Skill has none of these; it has no identity to score or isolate.
- The recommended structure: each Domain Mapping Agent is a persistent Claude agent (via the Agent SDK) that calls a shared library of Skills as tools for common extraction/classification sub-tasks, while the agent itself owns the domain scope, the tuning corpus, and the registry entry. This gives you reuse (via Skills) without sacrificing per-domain accountability (via the agent registry).

This resolves the ambiguity in the original framing: the reusable procedural knowledge can live in Skills; the thing that is 'tuned and trained using domain-specific knowledge' and can be individually monitored for drift has to be a registered agent, because only a registered agent has a persistent identity for MTBH and Value-Drift scoring to attach to.

5.3 Component Reuse Strategy: Chassis vs. Fully Bespoke Agents

Section 3.1 splits every agent into a shared chassis and an isolated cartridge. That split is itself a decision with alternatives worth recording:

Option	Reuse	Isolation	Assessment
A. Fully bespoke per-domain codebase (no shared chassis)	None -- every domain reimplements retrieval, LLM calling, telemetry	Strong by construction, but accidentally: nothing forces it	Rejected -- a chassis bug fix would need to be applied N times, and drift monitoring would differ subtly by domain
B. Shared chassis + isolated cartridge (chosen)	High -- six chassis components maintained once, used by every agent	Structural -- cartridges cannot merge; retraining one never touches another	Selected -- matches the reuse-and-isolation requirement directly, and is how the pilot in the companion code is built
C. One shared agent with per-domain configuration flags	Highest -- a single running process for all domains	Weakest -- a config bug or a bad flag can affect every domain simultaneously	Rejected -- this is Option A from Section 5.1 wearing a component-design label; it reintroduces the single-point-of-failure risk the domain-sovereignty requirement rules out

Option B's chassis is owned centrally (Semantic Governance Center of Excellence); its cartridges are owned by each Domain Governance Team. That ownership split mirrors the existing L1/L2/L4 governance-tier structure already defined in the source architecture, rather than inventing a new one.

6. Open Questions and What to Revisit

- GraphDB vendor and query interface (SPARQL vs. Cypher vs. GraphQL) — affects how the validation gate and reconciliation stage are implemented, not the architecture itself.
- Embedding model and vector DB vendor per domain — can differ by domain if domains have very different content types (e.g., a legal domain vs. a sensor-telemetry domain).
- Thresholds for MTBH, Value-Drift, and Intent Delta that trigger a circuit-breaker trip — these should be set per domain by the Domain Governance Team, not globally, and will need an initial calibration period.
- Ownership of the initial tuning corpus and few-shot examples for each domain agent — likely the same Domain Governance Team and SMEs who already own that domain's ontology, but this should be confirmed per domain.
- Whether Tier 1 (operational, <5s) mappings should be allowed to skip full cross-domain reconciliation, and what the fallback is if a fast-path mapping is later found to need reconciliation.
- As new artifact categories emerge beyond the initial nine, who owns adding the row to the content-type mapping table in Section 3.3 — likely the Semantic Governance Center of Excellence, per its existing platform-operations remit.
- Which of the seven roster agents in Section 3.2 to build first — likely whichever domain has the largest immediate backlog of ungoverned content, but this is a prioritization call, not an architectural one.

Appendix: Alignment to the Holonic Semantic Governance Architecture

This design introduces no new holonic level and no new governance primitive. It is a direct instantiation of the Level 2 Domain Holon pattern, adding one new component (the Domain Mapping Agent) to the existing domain holon composition, and one new operational concern (semantic-to-metadata mapping) to the existing RDSG traversal.

This Design's Component	Existing Architecture Counterpart	Holonic Level
Domain Mapping Agent	Domain Agent (L2 Domain Holon component)	L2
Agent chassis (shared, six components)	Holarchic protocol implementation -- shared RDSG-class infrastructure	L3
Agent cartridge (per domain, four components)	Domain Holon composition -- ontology + agent tuning + vector DB, per L2 definition	L2
Content & context router	RDSG Stage 0 — RCB Assembly	L3
Deterministic validation gate	RDSG Steps 1–3 (SHACL, SWRL, PEP)	L3
Cross-domain reconciliation	STG environment holonic composition	L2/L3
KGCL commit	ORSD/KGCL change protocol	L1
Mapping assertion node (reified relation)	KGCL change record, modeled as a graph node with about/resolvesTo edges	L1
Drift & governance monitor	MTBH dimensional scoring + Circuit Breaker	L3
Agent registry entry per agent	Domain Agent Registry	L2

— End of Document —